

COMPILADOR

Suaíli → Python

Documentação Técnica

Disciplina: Teoria da Computação e Compiladores

Linguagem-fonte: Suaíli customizado

Linguagem-alvo: Python 3

2026

Rodrigo Balthazar - RA:12725214410 , Samuel Peixoto - RA:1272318315
e Vitor Reis - RA:12722124265.

Compilador Suaíli

Documentação técnica apresentada ao curso de Ciência da Computação, na Unifacs, como parte dos requisitos necessários à aprovação do projeto final, A3.

Orientador: Eduardo Sidney da Silva Xavier.

Salvador-BA, 16/06/2026

1. Introdução

Este documento descreve o compilador desenvolvido para a disciplina de Linguagens Formais e Compiladores. O projeto consiste em um compilador que converte programas escritos em uma linguagem customizada baseada no idioma **Suaíli** para código **Python 3** válido e executável.

A Língua Suaíli é utilizada oficialmente o Quênia, Tanzânia e Uganda, porém é bastante utilizada em toda a região litoral sul, da Somália até Moçambique, como a República Democrática do Congo, Zâmbia, Ruanda e no sul da Etiópia, tendo cinquenta milhões de falantes no mundo, na Tanzânia e Quênia o idioma é bastante utilizado maioria de sua população tem conhecimento léxico, sendo referenciado em vários setores na educação, na mídia e na literatura.

1.1 Objetivos

- Definir uma linguagem de programação com palavras-chave em Suaíli;
- Implementar análise léxica (Lexer) completa;
- Implementar análise sintática (Parser) com gramática livre de contexto sem recursão à esquerda;
- Realizar análise semântica com verificação de tipos e escopo;
- Gerar código Python 3 correto e executável a partir do código-fonte.

1.2 Tecnologias Utilizadas

Componente	Tecnologia	Descrição
Compilador	JavaScript (ES6)	Lexer, Parser e Gerador de código
Interface	HTML/CSS/JS	Editor interativo no navegador
Linguagem-alvo	Python 3	Código gerado e executado

2. Definição da Linguagem Suaíli

A linguagem foi projetada para demonstrar os princípios de construção de compiladores utilizando palavras-chave em Suaíli, o idioma mais falado do leste africano. A sintaxe é imperativa, estruturada em blocos delimitados por chaves, e segue convenções próximas às linguagens C e Java.

2.1 Tipos de Dados

Palavra-chave	Tipo	Python equivalente	Exemplo
nambari	Inteiro	int	nambari x = 10
desimali	Ponto flutuante	float	desimali pi = 3.14
maneno	String	str	maneno s = "Ola"

Palavra-chave	Tipo	Python equivalente	Exemplo
kweli_uongo	Booleano	bool	kweli_uongo b = kweli

Valores booleanos literais: kweli (True) e uongo (False).

2.2 Palavras-chave Completas

Suaíli	Categoria	Equivalente	Python gerado
nambari	Tipo	inteiro	int
desimali	Tipo	decimal	float
maneno	Tipo	string	str
kweli_uongo	Tipo	booleano	bool
kweli / uongo	Literal bool	True/False	True/False
kama	Condicional	se	if
sivyo	Condicional	senão	else
wakati	Repetição	enquanto	while
fanya	Repetição	faça	while True
kwa	Repetição	para	while
chapisha	E/S	imprimir	print()
soma	E/S	ler string	input()
soma_nambari	E/S	ler inteiro	int(input())
soma_desimali	E/S	ler decimal	float(input())
na	Operador lógico	e	and
au	Operador lógico	ou	or
si	Operador lógico	não	not

3. Gramática da Linguagem

A gramática é livre de recursão à esquerda e não possui produções vazias (epsilon), conforme exigido pela especificação. Utiliza-se a notação BNF estendida.

3.1 Gramática BNF

Programa e Comandos

```

programa      ::= stmt+
stmt          ::= decl_stmt
               | assign_stmt
               | if_stmt
               | while_stmt
               | dowhile_stmt

```

```
| for_stmt
| print_stmt
| input_stmt
```

Declaração e Atribuição

```
decl_stmt ::= tipo ID '=' expr ';' '?'
          | tipo ID ';' '?'
assign_stmt ::= ID '=' expr ';' '?'
tipo ::= 'nambari' | 'desimali' | 'maneno' | 'kweli_uongo'
```

Estruturas de Controle

```
if_stmt ::= 'kama' '(' expr ')' block ( 'sivyo' ( if_stmt | block ) )?
while_stmt ::= 'wakati' '(' expr ')' block
dowhile_stmt ::= 'fanya' block 'wakati' '(' expr ')' ';' '?'
for_stmt ::= 'kwa' '(' for_init ';' expr ';' assign_stmt ')' block
for_init ::= tipo ID '=' expr | ID '=' expr
```

E/S e Bloco

```
print_stmt ::= 'chapisha' '(' expr ')' ';' '?'
input_stmt ::= ('soma'|'soma_nambari'|'soma_desimali') '(' ID ')' ';' '?'
block ::= '{' stmt* '}'
```

3.2 Precedência de Expressões

A precedência é garantida pela hierarquia das produções (maior prioridade no final da cadeia):

Nível	Produção	Operadores	Associatividade
1 (menor)	exprOr	au (or)	Esquerda
2	exprAnd	na (and)	Esquerda
3	exprNot	si (not)	Prefixo
4	exprCmp	== != < > <= >=	Esquerda
5	exprAdd	+ -	Esquerda
6	exprMul	* / %	Esquerda
7	exprUnary	- (unário)	Prefixo
8 (maior)	exprPrimary	literal, ID, ()	—

4. Arquitetura do Compilador

O Compilador é organizado em três fases sequenciais, todas implementadas em JavaScript puro no lado do cliente:

4.1 Fase 1 — Análise Léxica (Lexer)

O Lexer converte o texto-fonte em uma sequência de tokens. Cada token possui um tipo e um valor opcional.

Responsabilidades: Reconhecer números inteiros e decimais, strings delimitadas por aspas duplas, identificadores e palavras-chave, operadores de um e dois caracteres, e ignorar comentários de linha iniciados com `//` e espaços em branco.

Tokens reconhecidos

Categoria	Exemplos de lexemas	Token gerado
Tipos	nambari, desimali, maneno, kweli_uongo	INT, FLOAT, STRING, BOOL
Literais bool	kweli, uongo	BOOL_LIT
Inteiro	0, 42, 100	NUM_INT
Decimal	3.14, 0.5, 2.718	NUM_FLOAT
String	"Ola", "teste"	STR_LIT
Identificador	x, contador, resultado	ID
Operadores	+ - * / % == != < > <= >=	PLUS, MINUS, EQ, ...
Lógicos	na, au, si	AND, OR, NOT
Delimitadores	() { } ; ,	LPAREN, LBRACE, ...

4.2 Fase 2 — Análise Sintática e Semântica (Parser)

O Parser implementa um analisador descendente recursivo (Recursive Descent Parser). Cada produção da gramática corresponde a uma função JavaScript. A análise semântica é realizada embutida no parser.

Análise Semântica

- Tabela de símbolos implementada como pilha de escopos (um escopo por bloco {});
- Verificação de declaração antes do uso: toda variável deve ser declarada antes de ser referenciada;
- Verificação de escopo léxico: variáveis declaradas dentro de um bloco não são visíveis fora dele;
- Anotação de tipos: o tipo de cada variável é armazenado na tabela de símbolos e utilizado para geração de código.

4.3 Fase 3 — Geração de Código Python

O gerador emite código Python 3 diretamente durante a análise sintática. Utiliza a indentação via espaços, 4 por nível, para respeitar a sintaxe baseada em indentação do Python.

Mapeamentos principais

Construção Suaíli	Python gerado
<code>nambari x = 10</code>	<code>x: int = 10</code>
<code>desimali y = 3.14</code>	<code>y: float = 3.14</code>
<code>kama (x > 0) { ... }</code>	<code>if x > 0:</code>
<code>sivyo { ... }</code>	<code>else:</code>
<code>wakati (cond) { ... }</code>	<code>while cond:</code>
<code>fanya { ... } wakati (cond)</code>	<code>while True: ... if not (cond): break</code>
<code>kwa (tipo i=0; i<n; i=i+1)</code>	<code>i = 0while i < n: ... i = i + 1</code>
<code>chapisha(x)</code>	<code>print(x)</code>
<code>soma_nambari(x)</code>	<code>x = int(input())</code>
<code>soma_desimali(x)</code>	<code>x = float(input())</code>

5. Exemplos de Programas

5.1 Olá Mundo

Código Suaíli

```
// Olá Mundo em Kiswahili
maneno saudacao = "Jambo Dunia!"
chapisha(saudacao)
nambari x = 42
chapisha(x)
```

Python gerado

```
saudacao: str = "Jambo Dunia!"
print(saudacao)
x: int = 42
print(x)
```

5.2 Fatorial com for

Código Suaíli

```
nambari n = 5
nambari fat = 1
kwa (nambari i = 1; i <= n; i = i + 1) {
    fat = fat * i
}
```

```
chapisha(fat)
```

Python gerado

```
n: int = 5
fat: int = 1
i = 1
while i <= n:
    fat = fat * i
    i = i + 1
print(fat)
```

5.3 Par ou Ímpar (if/else + input)

Código Suaíli

```
nambari num = 0
soma_nambari(num)
kama (num % 2 == 0) {
    chapisha("par")
} sivyo {
    chapisha("impar")
}
```

Python gerado

```
num: int = 0
num = int(input())
if num % 2 == 0:
    print("par")
else:
    print("impar")
```

5.4 Fibonacci com while

Código Suaíli

```
nambari a = 0
nambari b = 1
nambari cont = 10
wakati (cont > 0) {
    chapisha(a)
    nambari temp = a + b
    a = b
    b = temp
    cont = cont - 1
}
```

Python gerado

```
a: int = 0
b: int = 1
cont: int = 10
while cont > 0:
    print(a)
    temp: int = a + b
    a = b
    b = temp
    cont = cont - 1
```

5.5 Do-While com tipos mistos

Código Suaíli

```
nambari i = 0
desimali soma = 0.0
kweli_uongo ativo = kweli
fanya {
    soma = soma + 1.5
    i = i + 1
} wakati (i < 4)
chapisha(soma)
chapisha(ativo)
```

Python gerado

```
i: int = 0
soma: float = 0.0
ativo: bool = True
while True:
    soma = soma + 1.5
    i = i + 1
    if not (i < 4):
        break
print(soma)
print(ativo)
```

6. Verificações Semânticas

O compilador realiza as seguintes verificações durante a análise:

6.1 Declaração antes do uso

Toda variável deve ser declarada com seu tipo antes de ser utilizada em qualquer expressão ou atribuição. Caso contrário, o compilador lança um erro com a mensagem:

```
ERRO: Variável não declarada: 'nome_da_variavel'
```

6.2 Escopo léxico

O Compilador mantém uma pilha de tabelas de símbolos, uma por bloco delimitado por chaves. Ao entrar em um bloco, um novo escopo é empilhado; ao sair, é removido da pilha. A busca de variáveis percorre a pilha do topo para a base, implementado corretamente o escopo léxico aninhado.

6.3 Checagem de tipos nas operações de input

Os três comandos de leitura garantem a conversão de tipo correspondente: soma gera `input()`, soma_nambari gera `int(input())` e soma_desimali gera `float(input())`.

7. Operadores

7.1 Operadores Aritméticos

Símbolo	Operação	Exemplo Kiswahili	Python
+	Adição	$x + y$	$x + y$
-	Subtração	$x - y$	$x - y$
*	Multiplicação	$x * y$	$x * y$
/	Divisão	x / y	x / y
%	Módulo	$x \% y$	$x \% y$

7.2 Operadores Relacionais

Símbolo	Significado	Exemplo	Python
==	Igual a	$x == y$	$x == y$
!=	Diferente de	$x != y$	$x != y$
<	Menor que	$x < y$	$x < y$
>	Maior que	$x > y$	$x > y$
<=	Menor ou igual	$x <= y$	$x <= y$
>=	Maior ou igual	$x >= y$	$x >= y$

7.3 Operadores Lógicos

Suaíli	Operação	Python
na	Conjunção (AND)	and
au	Disjunção (OR)	or

Suaíli	Operação	Python
si	Negação (NOT)	not

8. Mensagens de Erro

O compilador emite mensagens de erro descritivas para as situações a seguir:

Situação	Mensagem de erro
Variável usada sem declaração	Variável não declarada: 'nome'
Token inesperado no parser	Esperado TIPO, encontrado TIPO ('val')
Expressão inválida	Expressão inválida em: TIPO 'val'
Caractere não reconhecido	Caractere desconhecido: 'c' na posição N

9. Referências

- PYPI, Pyswahili. Disponível em:< <https://pypi.org/project/pyswahili/>>. Acesso em 28/05/2026;
- PYTHON SOFTWARE FOUNDATION. Python 3 Documentation. Disponível em: <<https://docs.python.org/3/>>. Acesso em 28/05/2026;
- AHO, A. V. et al. Compiladores: Princípios, Técnicas e Ferramentas. 2. ed. Pearson, 2008;
- COOPER, K.; TORCZON, L. Engineering a Compiler. 2. ed. Morgan Kaufmann, 2011;
- LOUDEN, K. C. Compiladores: Princípios e Práticas. Cengage Learning, 2004.

Fim do documento.